

a. Name of two GenAI programs

The 2 AI's that I used for this assignment are Gemini and ChatGPT.

b. How you accessed them (e.g., Google query, CoPilot on my computer)

I accessed both of these AI through both of their websites

c. Prompt used. (You must use the same prompt for both GenAIs.)

The program will prioritize emails for a busy company CEO.

- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap can handle all your email prioritizing for the CEO.
- You must create functions from scratch. Do not include pre-existing heap modules.

- Test File format:

Command Description

EMAIL <sender category>,

<subject line>,

<date>

The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces, but not commas

Assume <date> is in the format: MM-DD-YYYY

NEXT

Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:

Subject:

Date:

READ CEO has read email and dealt with it

COUNT Display current count of unread emails

- The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

- Sample Test File:

EMAIL Peer,Can you help me on this?,12-01-2024

EMAIL OtherPerson,Try our product,12-19-2024

EMAIL Boss,Important,12-20-2024

EMAIL Subordinate,How do I handle this?,12-25-2024

EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024

EMAIL Boss,Never Mind,01-03-2025

COUNT

NEXT

READ

NEXT

READ

COUNT

- Your program should display this:

There are 6 emails to read.

Next email:

Sender: Boss

Subject: Never Mind

Date: 01-03-2025

Next email:

Sender: Boss

Subject: Important

Date: 12-20-2024

There are 4 emails to read.

d. Generated code from both GenAIs.

Chat GPT Code:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#define MAX_EMAILS 1000
```

```
/* Structure to store an email */
```

```
typedef struct {  
    char sender[30];
```

```

    char subject[200];
    char date[11];

    int priority;
    int dateValue;
} Email;

/* Structure for the MaxHeap */
typedef struct {
    Email emails[MAX_EMAILS];
    int size;
} MaxHeap;

/* -----
HELPER FUNCTIONS
----- */

/* Convert sender category into a priority number */
int getPriority(char sender[]) {
    if (strcmp(sender, "Boss") == 0)
        return 5;
    else if (strcmp(sender, "Subordinate") == 0)
        return 4;
    else if (strcmp(sender, "Peer") == 0)
        return 3;
    else if (strcmp(sender, "ImportantPerson") == 0)
        return 2;
    else
        return 1; /* OtherPerson */
}

/*
Convert MM-DD-YYYY into a number that can
be compared chronologically.

Example:
01-03-2025 becomes 20250103
12-20-2024 becomes 20241220

A larger number means a newer date.
*/
int getDateValue(char date[]) {

```

```

    int month, day, year;

    sscanf(date, "%d-%d-%d", &month, &day, &year);

    return year * 10000 + month * 100 + day;
}

/*
    Returns 1 if email1 should be ahead of email2
    in the MaxHeap.
*/
int higherPriority(Email email1, Email email2) {

    /* First compare sender category */
    if (email1.priority > email2.priority)
        return 1;

    if (email1.priority < email2.priority)
        return 0;

    /* Same sender category: newer email comes first */
    if (email1.dateValue > email2.dateValue)
        return 1;

    return 0;
}

/* -----
    MAX HEAP FUNCTIONS
    ----- */

/* Swap two emails */
void swap(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

/* Initialize the heap */
void initializeHeap(MaxHeap *heap) {
    heap->size = 0;

```

```
}
```

```
/*
```

```
    Insert an email into the MaxHeap.
```

```
    The new email is placed at the bottom,  
    then moved upward until the heap property  
    is restored.
```

```
*/
```

```
void insert(MaxHeap *heap, Email email) {
```

```
    if (heap->size >= MAX_EMAILS) {  
        return;  
    }
```

```
    int index = heap->size;
```

```
    heap->emails[index] = email;  
    heap->size++;
```

```
    /* Move the new email upward */
```

```
    while (index > 0) {
```

```
        int parent = (index - 1) / 2;
```

```
        if (higherPriority(heap->emails[index],  
                           heap->emails[parent])) {
```

```
            swap(&heap->emails[index],  
                 &heap->emails[parent]);
```

```
            index = parent;
```

```
        }
```

```
        else {  
            break;
```

```
        }
```

```
    }
```

```
}
```

```
/*
```

```
    Return the highest-priority email without  
    removing it.
```

```

*/
Email *peek(MaxHeap *heap) {

    if (heap->size == 0) {
        return NULL;
    }

    return &heap->emails[0];
}

/*
    Remove the highest-priority email.

    The last email is moved to the root,
    then moved downward until the heap property
    is restored.
*/
void removeMax(MaxHeap *heap) {

    if (heap->size == 0) {
        return;
    }

    /* Move last element to root */
    heap->emails[0] = heap->emails[heap->size - 1];

    heap->size--;

    int index = 0;

    /* Move the root downward */
    while (1) {

        int left = 2 * index + 1;
        int right = 2 * index + 2;
        int largest = index;

        /* Check left child */
        if (left < heap->size &&
            higherPriority(heap->emails[left],
                          heap->emails[largest])) {

            largest = left;

```

```

    }

    /* Check right child */
    if (right < heap->size &&
        higherPriority(heap->emails[right],
                      heap->emails[largest])) {

        largest = right;
    }

    /* If the current node is already largest,
       the heap is fixed. */
    if (largest == index) {
        break;
    }

    swap(&heap->emails[index],
        &heap->emails[largest]);

    index = largest;
}
}

/* -----
EMAIL FUNCTIONS
----- */

/*
Create an Email structure from the three
pieces of information.
*/
Email createEmail(char sender[], char subject[], char date[]) {

    Email email;

    strcpy(email.sender, sender);
    strcpy(email.subject, subject);
    strcpy(email.date, date);

    email.priority = getPriority(sender);
    email.dateValue = getDateValue(date);

    return email;
}

```

```
}
```

```
/*
```

```
    Display the highest-priority email without  
    removing it from the heap.
```

```
    This is used for NEXT.
```

```
*/
```

```
void nextEmail(MaxHeap *heap) {
```

```
    Email *email = peek(heap);
```

```
    if (email == NULL) {  
        return;  
    }
```

```
    printf("Next email:\n");  
    printf("Sender: %s\n", email->sender);  
    printf("Subject: %s\n", email->subject);  
    printf("Date: %s\n", email->date);
```

```
}
```

```
/*
```

```
    READ removes the highest-priority email.
```

```
    Nothing is displayed.
```

```
*/
```

```
void readEmail(MaxHeap *heap) {  
    removeMax(heap);  
}
```

```
/*
```

```
    Display the current number of unread emails.
```

```
*/
```

```
void countEmails(MaxHeap *heap) {
```

```
    printf("There are %d emails to read.\n", heap->size);
```

```
}
```

```
/* -----
```

MAIN

```
----- */

int main(void) {

    MaxHeap heap;

    initializeHeap(&heap);

    char line[300];

    /*
    Read the test file one line at a time.
    The grader can redirect a file into the program,
    for example:

    ./program < test.txt
    */
    while (fgets(line, sizeof(line), stdin) != NULL) {

        /* Remove newline character */
        line[strcspn(line, "\n")] = '\0';

        /* -----
        EMAIL command
        ----- */
        if (strncmp(line, "EMAIL ", 6) == 0) {

            char sender[30];
            char subject[200];
            char date[11];

            char *data = line + 6;

            /*
            Find the first comma.
            Everything before it is the sender.
            */
            char *firstComma = strchr(data, ',');

            if (firstComma == NULL) {
                continue;
            }
        }
    }
}
```

```

*firstComma = '\0';

strcpy(sender, data);

/*
    Find the second comma.
    Everything between the first and second
    comma is the subject.
*/
char *secondComma = strchr(firstComma + 1, ',');

if (secondComma == NULL) {
    continue;
}

*secondComma = '\0';

strcpy(subject, firstComma + 1);
strcpy(date, secondComma + 1);

/* Create the email */
Email email = createEmail(sender, subject, date);

/* Put it into the MaxHeap */
insert(&heap, email);
}

/* -----
    NEXT command
    ----- */
else if (strcmp(line, "NEXT") == 0) {

    nextEmail(&heap);
}

/* -----
    READ command
    ----- */
else if (strcmp(line, "READ") == 0) {

```

```

        readEmail(&heap);
    }

    /* -----
    COUNT command
    ----- */
    else if (strcmp(line, "COUNT") == 0) {

        countEmails(&heap);
    }
}

return 0;
}

```

Gemini Code:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_LINE_LEN 512
#define MAX_STR_LEN 256
#define INITIAL_CAPACITY 16

typedef struct {
    char sender[MAX_STR_LEN];
    char subject[MAX_STR_LEN];
    char date[MAX_STR_LEN];
    int sender_priority;
    long date_val;    // Formatted as YYYYMMDD for direct chronological comparison
} Email;

typedef struct {
    Email *arr;
    int size;
    int capacity;
} MaxHeap;

// Helper to assign numeric weight to categories: Higher number = Higher priority
int get_category_priority(const char *category) {
    if (strcmp(category, "Boss") == 0) return 5;
    if (strcmp(category, "Subordinate") == 0) return 4;
    if (strcmp(category, "Peer") == 0) return 3;

```

```

    if (strcmp(category, "ImportantPerson") == 0) return 2;
    if (strcmp(category, "OtherPerson") == 0) return 1;
    return 0;
}

// Convert "MM-DD-YYYY" into integer YYYYMMDD for straightforward numeric comparison
long parse_date(const char *date_str) {
    int m, d, y;
    if (sscanf(date_str, "%d-%d-%d", &m, &d, &y) == 3) {
        return (long)y * 10000 + (long)m * 100 + (long)d;
    }
    return 0;
}

// Returns > 0 if a has higher priority than b, < 0 if b is higher, 0 if equal
int compare_emails(const Email *a, const Email *b) {
    if (a->sender_priority != b->sender_priority) {
        return a->sender_priority - b->sender_priority;
    }
    // Tie-breaker: newest date takes precedence
    if (a->date_val > b->date_val) return 1;
    if (a->date_val < b->date_val) return -1;
    return 0;
}

// Heap Operations
MaxHeap* create_heap() {
    MaxHeap *h = (MaxHeap *)malloc(sizeof(MaxHeap));
    h->size = 0;
    h->capacity = INITIAL_CAPACITY;
    h->arr = (Email *)malloc(sizeof(Email) * h->capacity);
    return h;
}

void swap(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

void heapify_up(MaxHeap *h, int idx) {
    while (idx > 0) {
        int parent = (idx - 1) / 2;
        if (compare_emails(&h->arr[idx], &h->arr[parent]) > 0) {

```

```

        swap(&h->arr[idx], &h->arr[parent]);
        idx = parent;
    } else {
        break;
    }
}
}

```

```

void heapify_down(MaxHeap *h, int idx) {
    while (1) {
        int left = 2 * idx + 1;
        int right = 2 * idx + 2;
        int largest = idx;

        if (left < h->size && compare_emails(&h->arr[left], &h->arr[largest]) > 0) {
            largest = left;
        }
        if (right < h->size && compare_emails(&h->arr[right], &h->arr[largest]) > 0) {
            largest = right;
        }

        if (largest != idx) {
            swap(&h->arr[idx], &h->arr[largest]);
            idx = largest;
        } else {
            break;
        }
    }
}

```

```

void insert_email(MaxHeap *h, Email e) {
    if (h->size == h->capacity) {
        h->capacity *= 2;
        h->arr = (Email *)realloc(h->arr, sizeof(Email) * h->capacity);
    }
    h->arr[h->size] = e;
    heapify_up(h, h->size);
    h->size++;
}

```

```

Email* peek_email(MaxHeap *h) {
    if (h->size == 0) return NULL;
    return &h->arr[0];
}

```

```

void pop_email(MaxHeap *h) {
    if (h->size == 0) return;
    h->arr[0] = h->arr[h->size - 1];
    h->size--;
    if (h->size > 0) {
        heapify_down(h, 0);
    }
}

```

```

void free_heap(MaxHeap *h) {
    if (h) {
        free(h->arr);
        free(h);
    }
}

```

// String helper to trim leading/trailing whitespaces and newlines

```

void trim(char *str) {
    int start = 0;
    while (str[start] == ' ' || str[start] == '\t' || str[start] == '\r' || str[start] == '\n') {
        start++;
    }
    int end = strlen(str) - 1;
    while (end >= start && (str[end] == ' ' || str[end] == '\t' || str[end] == '\r' || str[end] == '\n')) {
        end--;
    }
    int len = end - start + 1;
    if (len > 0) {
        memmove(str, str + start, len);
        str[len] = '\0';
    } else {
        str[0] = '\0';
    }
}

```

```

int main(int argc, char *argv[]) {
    FILE *file = stdin;
    if (argc > 1) {
        file = fopen(argv[1], "r");
        if (!file) {
            fprintf(stderr, "Error: Could not open file %s\n", argv[1]);
            return 1;
        }
    }
}

```

```
}
```

```
MaxHeap *heap = create_heap();
```

```
char line[MAX_LINE_LEN];
```

```
while (fgets(line, sizeof(line), file)) {
```

```
    trim(line);
```

```
    if (strlen(line) == 0) continue;
```

```
    if (strncmp(line, "EMAIL ", 6) == 0) {
```

```
        char *content = line + 6;
```

```
        char *sender = strtok(content, ",");
```

```
        char *subject = strtok(NULL, ",");
```

```
        char *date = strtok(NULL, ",");
```

```
        if (sender && subject && date) {
```

```
            trim(sender);
```

```
            trim(subject);
```

```
            trim(date);
```

```
            Email e;
```

```
            strncpy(e.sender, sender, MAX_STR_LEN - 1);
```

```
            e.sender[MAX_STR_LEN - 1] = '\0';
```

```
            strncpy(e.subject, subject, MAX_STR_LEN - 1);
```

```
            e.subject[MAX_STR_LEN - 1] = '\0';
```

```
            strncpy(e.date, date, MAX_STR_LEN - 1);
```

```
            e.date[MAX_STR_LEN - 1] = '\0';
```

```
            e.sender_priority = get_category_priority(e.sender);
```

```
            e.date_val = parse_date(e.date);
```

```
            insert_email(heap, e);
```

```
        }
```

```
    } else if (strcmp(line, "NEXT") == 0) {
```

```
        Email *next_email = peek_email(heap);
```

```
        if (next_email != NULL) {
```

```
            printf("Next email:\n");
```

```
            printf("Sender: %s\n", next_email->sender);
```

```
            printf("Subject: %s\n", next_email->subject);
```

```
            printf("Date: %s\n", next_email->date);
```

```
        }
```

```
    } else if (strcmp(line, "READ") == 0) {
```

```
        pop_email(heap);
```

```
    } else if (strcmp(line, "COUNT") == 0) {
```

```

        printf("There are %d emails to read.\n", heap->size);
    }
}

if (file != stdin) {
    fclose(file);
}
free_heap(heap);
return 0;
}

```

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.

Chat GPT Code:

- Correctness
 - Program runs correctly and has the correct output
 - The code seems to be able to run commands such as “READ” and “CHECK” even if they aren’t capitalized, meaning that it has a function that makes them uppercase
 - Code also doesn’t error if I put in a random string such as “HI”, meaning that the code is able to ignore unnecessary outputs without crashing
 - Although If i remove commas from the emails inputted, the program doesn’t consider the lines as emails,
- Execution Time
 - Chat GPT mainly consists of single loops, but specifically I only found 3 while loops in the entire code, meaning it has $O(n)$ complexity
 - Doesn’t have any recursive calls In the program
 - The function insert() and removeMax() has a complexity of $O(\log n)$, since both functions change the order of the emails within the heap or call stacks
- Space Complexity
 - The overall space complexity of the code is $O(n)$, mainly due to the fact that the Max Heap that is created is used to store emails and is proportionally sized based on how many emails are added, hence the code max complexity is $O(n)$.
- Maintainability
 - Overall, the code is very readable and quite easy to understand, with lots of code lines in between functions describing the purpose of each function properly and it has variable names that make sense

Gemini Code:

- Correctness
 - Code produces the required output

- When i change the output to lowercase for “CHECK”, it doesn’t count it as a command since it doesn’t uppercase it when reading the output
 - Doesn’t crash when given a random number or string
 - Also doesn’t take in an email if commas are removed from the emails
- Execution Time
 - Execution Time is fairly similar to Chat GPTs code, where the insert() and remove functions(insert_email and pop_email in Gemini) both have $O(\log n)$ complexity, since both functions go through the email list the same way, therefore their execution time is identical.
- Space Complexity
 - The gemini code also has $O(n)$ space complexity, since it also only has the heap storage to worry about, which the allowed capacity for it will never exceed more than $2N$.
- Maintainability
 - A lot harder to read the code, there are way less comments that explain what each part of the code does. It’s way more broad and doesn’t really break down what each function does.

f. Select one to use as the basis for this assignment and justify why you picked it.

For the code, I decided to use ChatGPTs response to it because although both codes are very similar to each other, I think that GPTs is way more readable and the naming conventions that the LLM decided to use makes way more sense than what Gemini uses, so It’s a lot easier to work with than Geminis code.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

- Correctness: Nothing particularly changed in terms of correctness. The given code didn’t have any errors on runtime so the main thing is that I added a limit to the heap stack In case any errors occurring from too many emails
- Execution time: I separated some of the processing which reduced the amount of functions running in the main(), allowing for less executing time on runtime
- Space complexity: No change to space complexity, since $O(n)$ is the most efficient space complexity that you could make for this program
- Maintainability: Created more accurate enums for the priority of each email, allowing for more specific naming conventions and with the use of the priority function. Also, I included safer input parsing using strncpy().